`mb_ereg_replace()` – and the case-insensitive form `mb_eregi_replace()` – are pretty much the same as `preg_replace()` but do not handle arrays of sample strings, replacements, or patterns. They only deal with single strings but do accept a string of behavior modifiers and a mode (see `mb_regex_set_options()` below).

```
$text = "\u{142}ukasz";
print $text . PHP_EOL;
print mb_ereg_replace('\x{142}', "\u{141}", $text);
```

```
Output:
łukasz
Łukasz
```

Note the difference between Unicode escapes in strings and regex patterns: `\u{nnn}` for strings and `\x{nnn}` for the pattern. Case sensitivity is based on the Unicode data files, so no matter which character set is used, you should get correct regional equivalence:

```
print mb_eregi_replace('(\x{141}\w+)', 'My buddy \1', $text);
```

```
Output:
My buddy łukasz
```

Here we're searching for an uppercase `'Ł'` (U+0141) in the pattern but running in case-insensitive mode. The Unicode data files that are used to compile `mbstring` map U+0141 and U+0142 as case equivalents, so the pattern matches even though it actually contains a U+0142 character. Also note that the backreference `\1` is enclosed in a single-quoted string (it becomes another kind of escape if double quotes are used).

`mb_ereg_replace_callback()` is the counterpart to `preg_replace_callback()`. Again, it does not support arrays of patterns or sample strings but does accept modifiers and mode parameters.

```
$text = "\u{c7}a\u{11f}ari";
print $text . PHP_EOL;
print mb_ereg_replace_callback(
    "\w+",
    function ($matches) {
        return mb_strtoupper($matches[0]);
    },
```